Proyecto Dossier / TraceLens — Estado de avance vs. alcance comprometido

Fecha: 31 de agosto de 2026

Base de comparación: ficha del proyecto (modalidad Continuidad) y Especificaciones v2.0, contrastadas con el código realmente entregado en `SistemaDossier-principal` (julio 2026).

Nota sobre los porcentajes: son estimaciones de juicio basadas en inspección de código, no en horas medidas. Sirven para priorizar, no para facturar.

---

1. Resumen ejecutivo

El proyecto está **más avanzado de lo que sugiere su documentación y menos completo de lo que promete su ficha**. Hay un sistema real, desplegado, multi-tenant, con dos integraciones de calendario funcionando y un motor de dossiers corporativos operativo. Eso es sustancial.

La brecha se concentra en tres frentes:

1. **La salida del dossier no es estructurada.** El brief promete «capa de síntesis LLM con schema JSON estricto» con ice breakers, alertas y línea de tiempo como entidades. Lo implementado produce **Markdown en prosa**. Las cinco tablas que darían estructura a esa salida están diseñadas y no existen en el código.
2. **Faltan los componentes de monetización y entrega.** Sin Stripe, sin envío por email y sin API pública, el modelo freemium descrito no puede operar: los créditos se asignan a mano.
3. **Hay tres compromisos de seguridad declarados y no cumplidos**, uno de ellos serio (tokens OAuth de calendario en texto plano).

Avance global estimado del MVP comprometido: 55–60 %.

Frente	Estado
Plataforma base (auth, multi-tenant, RBAC, BD)	🟢 85 %
Motor de dossiers corporativos (UK/USA)	🟢 75 %
Investigación de personas	🟡 60 %
Automatización de calendario	🟢 80 %
Estructuración de la salida (JSON, ice breakers, alertas)	🔴 20 %
Monetización (Stripe, créditos automáticos)	🔴 25 %
Entrega por email	🔴 0 %
Tiempo real (WebSocket)	🔴 0 %
Cobertura Europa continental	🔴 0 %
Auditoría y cumplimiento	🔴 15 %
Pruebas y CI/CD	🔴 10 %

---

2. Componentes: lo comprometido contra lo construido

Componente de la ficha	Estado	Detalle
Dashboard web	🟢 Hecho	Next.js 16, ~22.700 líneas TSX, tema claro/oscuro, i18n
Orquestador de agentes	🟡 Parcial	LangGraph con 3 nodos: UK, USA y síntesis
`Agent_Identity`	🟡 Parcial	Vía People Data Labs, no como nodo del grafo
`Agent_Corporate_UK`	🟢 Hecho	Companies House + lectura de formularios regulatorios
`Agent_Corporate_USA`	🟢 Hecho	SEC EDGAR + Company Facts
`Agent_Corporate_EU`	🔴 No existe	Sin registros europeos continentales

`Agent_News`	🔴 No existe	Sin Tavily ni SerpApi
`Agent_Social`	🔴 No existe	Descartado al abandonar el scraping
Síntesis LLM con schema JSON estricto	🔴 Parcial	Genera Markdown en prosa, sin schema
Automatización Google Calendar	🟢 Hecho	OAuth + worker + filtros por dominio/asistentes
Automatización Outlook	🟢 Hecho	Microsoft Graph, misma ruta
Entrega por email (Gmail/Outlook/SMTP)	🔴 No existe	Solo columnas en el DDL
Billing por créditos	🟡 Parcial	Descuento y saldos sí; sin Stripe ni libro mayor
RBAC	🟡 Parcial	4 roles planos, no permisos por recurso
Audit logging (estándar Alloxentric)	🔴 No existe	Tabla diseñada, cero escrituras

3. Stack: lo documentado contra lo implementado

Ficha	Realidad	Lectura
Next.js + React + TS + Tailwind	Next.js 16, React 19, Tailwind 4	**OK** Coincide
FastAPI + REST + JWT + multi-tenancy	Igual	**OK** Coincide
WebSocket	Polling con `setInterval`	**NO** No implementado
LangChain / LangGraph	LangGraph ≥1.2	**OK** Coincide (alcance reducido)
Síntesis con GPT-4o	**DeepSeek** (+ Gemini en CLIs, OpenAI tras flag)	**!** Cambio de proveedor no documentado
PostgreSQL	PostgreSQL + SQLAlchemy	**OK** Coincide
Redis para caché 24 h	Implementado	**OK** Coincide
Redis para rate limiting	No existe rate limiting	**NO** No implementado
Companies House	Implementado	**OK**
SEC EDGAR	Implementado	**OK**
OpenCorporates	No integrado	**NO**
Proxycurl / Apollo	**People Data Labs + Lusha**	**!** Cambio de proveedor no documentado
Tavily / SerpApi	No integrado	**NO**
Stripe	No integrado	**NO**
TLS 1.3	Provisto por Render/Vercel	**OK** Por plataforma
AES-256 en reposo	**Tokens OAuth en texto plano**	**NO** Ver §6

Los dos cambios de proveedor (GPT-4o → DeepSeek, Proxycurl → PDL) son decisiones **probablemente correctas** —DeepSeek es sensiblemente más barato y PDL evita el scraping—, pero no están documentadas en ningún lado. Escribirlas cuesta dos páginas y convierte un aparente incumplimiento en una decisión de ingeniería defendible.

4. El hallazgo estructural: 19 tablas diseñadas, 10 implementadas

`src/dossier/db/Migracion.md` contiene 53 KB de DDL con 19 tablas. `db/models.py` implementa 10.

Diseñadas e implementadas (7): `organizations`, `users`, `org_memberships`, `calendar_integrations`, `calendar_events`, `contacts`, `dossiers`

Implementadas sin estar en el diseño (3): `dossier_generation_jobs`, `dossier_shares`, `org_invites` — se agregaron durante el desarrollo y nunca se documentaron de vuelta

Diseñadas y ausentes (12):

Tabla ausente	Qué funcionalidad bloquea
`dossier_ice_breakers`	Los 3 ice breakers con fuente citada
`dossier_alerts`	Alertas de riesgo crítico/advertencia estructuradas
`dossier_career_entries`	Línea de tiempo profesional con detección de brechas

`dossier_corporate_records`	Tabla ordenable de cargos y sociedades
`dossier_media_mentions`	Menciones en prensa
`credit_ledger`	Trazabilidad de consumo de créditos
`billing_transactions`	Cobros vía Stripe
`email_integrations`	Entrega por email
`api_keys`	API pública Enterprise
`audit_logs`	Auditoría WHO/WHAT/WHEN/WHERE/RESULT
`refresh_tokens`	Revocación de sesión
`oauth_accounts`	Login social unificado

Las cinco primeras cuentan la misma historia: **el dossier se guarda como un bloque de texto Markdown, no como datos**. Eso explica por qué el frontend usa `react-markdown` para renderizarlo y por qué no hay tabla ordenable de Companies House ni línea de tiempo visual.

La consecuencia práctica va más allá de la interfaz: sin salida estructurada no se puede medir la tasa de alucinación (<5 % es un KPI comprometido), no se pueden verificar las URLs de las fuentes, no se puede exportar a CRM y no se puede construir la API pública.

---

5. El flujo prometido contra el flujo real

Paso del brief	Implementado
Configuración por checkboxes de módulos	**NO** Tres niveles fijos: basic / standard / deep
Costos 1 / 3 / 5 créditos	**OK** Exactamente así (`DEPTH_CREDITS`)
Disparo automático 15 min – 1 día antes	**OK** `advance_minutes`, por defecto 20
Agentes en paralelo	**OK** UK y USA en paralelo real desde `START`
Detección de discrepancias LinkedIn vs. registros	**!** Depende del prompt; sin comparación programática
Alertas crítico / advertencia	**!** Tres niveles (leve/moderada/crítica) como marcas de texto
Hasta 3 ice breakers con fuente citada	**NO** Sin entidad ni verificación de URL
Línea de tiempo profesional	**NO** No estructurada
Estados progresivos vía WebSocket	**NO** Polling
Resultados parciales si un agente falla	**OK** El grafo tolera un lado vacío
Caché 24 h y eliminación (GDPR)	**!** Caché Redis 24 h sí; el dossier se retiene 30 días por defecto y admite «nunca»

El punto sobre GDPR merece atención: el brief dice «se elimina (privacidad por diseño)». El sistema permite configurar retención en 7/14/30/90 días **o indefinida**. Es una decisión de producto razonable, pero contradice el compromiso escrito y, tratándose de perfiles de personas identificables, es exactamente el tipo de discrepancia que un cliente Enterprise auditaría.

---

6. Lo que hay que resolver antes de seguir construyendo

Tres compromisos de seguridad declarados en la ficha y no cumplidos:

S1 — Tokens OAuth de calendario almacenados en claro. `services/calendar_integrations.py:34` lo dice con todas las letras: *«Los campos `*_encrypted` guardan el valor en claro por ahora (MVP); cifrar en app antes»*. Un refresh token de Google Calendar da acceso persistente al calendario del cliente. Contradice directamente el «AES-256» de la ficha. Se resuelve con Fernet y una clave en variable de entorno: media jornada.

S2 — CORS abierto a cualquier `*.vercel.app`, activado por defecto. `api/app.py:196`. Cualquiera puede registrar un subdominio gratuito de Vercel y quedar autorizado contra la API de producción. Cambiar el valor por defecto a `0`: quince minutos.

S3 — Sin rate limiting ni bloqueo de cuenta en `/auth/*`. Sumado a que el registro responde 409 con mensaje explícito, se filtra qué correos tienen cuenta y no hay fricción para fuerza bruta. SlowAPI resuelve lo esencial en unas 20 líneas.

A esto se suma **la ausencia total de `audit_logs`**, que la Sección 6 de los Estándares Transversales define como «imperativo ético» y principio de no repudio. Para un producto que genera perfiles de personas y cobra por ello, no poder responder «quién consultó el dossier de X y cuándo» es la brecha de cumplimiento más costosa de arrastrar.

---

7. Lo que ya se lleva — inventario de logros

Conviene tenerlo escrito, porque no está en ningún documento y es lo que sostiene la continuidad:

- **Multi-tenancy real y correcto.** Toda consulta filtra por `organization_id`. Revisé `dossier_routes.py` completo, incluidas las rutas de compartición: no encontré una sola fuga entre organizaciones.
- **Cola de trabajos bien resuelta.** `with_for_update(skip_locked=True)` es la forma canónica de repartir trabajo entre réplicas sin duplicar. Está bien hecho.
- **OAuth con `state` firmado** que liga usuario + organización + propósito, con caducidad de 15 minutos y validación del campo `purpose`. Por encima del promedio.
- **Dos integraciones de calendario en producción** (Google y Microsoft Graph) con filtros por dominio, asistentes mínimos, reuniones internas y recurrentes.
- **Feature gating por plan** con 402/403 según corresponde, y créditos descontados por trigger en PostgreSQL.
- **i18n con seis idiomas** y prompts localizados (48 KB de plantillas por idioma), más tema claro/oscuro por tokens CSS. Dos de los cinco estándares transversales cumplidos.
- **Sin inyección SQL** en 17.500 líneas: SQLAlchemy usado correctamente en todas partes.
- **Retención configurable de dossiers** con worker de purga, más de lo que el diseño original contemplaba.

---

8. Lo que falta para cerrar el MVP comprometido

Ordenado por relación impacto/esfuerzo. Las estimaciones son de orden de magnitud.

Bloque A — Desbloquea el modelo de negocio (~4 semanas)

Tarea	Esfuerzo
Integrar Stripe: checkout, webhooks con idempotencia, portal de cliente	2 sem
`credit_ledger` + `billing_transactions` para trazabilidad	3 días
Envío por email (proveedor transaccional + plantillas + cola)	1 sem
Recuperación de contraseña sobre ese mismo servicio	2 días

Bloque B — Convierte el dossier en producto (~4 semanas)

Tarea	Esfuerzo
Salida estructurada: forzar JSON en la síntesis y validar con Pydantic	1 sem
Tablas `dossier_alerts`, `ice_breakers`, `career_entries`, `corporate_records`, `media_mentions`	3 días
Verificación de URLs de fuentes (anti-alucinación)	3 días
Renderizado en frontend: tabla ordenable, timeline, badges de alerta	1,5 sem
Dataset de evaluación para medir el KPI de <5 % de alucinación	4 días

Bloque C — Cumplimiento y confianza (~2 semanas)

Tarea	Esfuerzo
S1, S2, S3 (cifrado de tokens, CORS, rate limiting)	2 días
`audit_logs` + middleware + `REVOKE UPDATE, DELETE`	4 días
`refresh_tokens` y revocación de sesión; bajar expiración del JWT	3 días
Alinear política de retención con lo declarado, o corregir la declaración	1 día

Bloque D — Calidad de ingeniería (~2 semanas)

Tarea	Esfuerzo
25–30 tests de endpoint (auth, aislamiento, créditos, gating)	1 sem
GitHub Actions: ruff + pytest en cada push	1 día
Fijar versiones en `requirements.txt`; añadir pytest	2 h
Adoptar Alembic; llenar el `schema_project_dossier.sql` (hoy 0 bytes)	3 días
Reaper de jobs colgados con reembolso de créditos	2 días

Bloque E — Ampliación de alcance (posterior al MVP)

WebSocket o SSE en lugar de polling · configuración por checkboxes en vez de tres niveles · `Agent_News` con Tavily · registros corporativos de Europa continental · API pública con `api_keys` · white-label Enterprise.

Ruta crítica al MVP comprometido: aproximadamente 12 semanas con el equipo actual, ejecutando A y B en paralelo con C.

---

9. Recomendación

Tres cosas, en este orden:

Primero, cerrar el Bloque C esta semana. Son dos días de trabajo real y elimina el riesgo de tener que explicar por qué los tokens de calendario de un cliente estaban en claro. Todo lo demás puede esperar; esto no.

Segundo, decidir explícitamente sobre la salida estructurada. Es la bifurcación arquitectónica del proyecto. Si el dossier sigue siendo Markdown, hay que renegociar el KPI de alucinación, la tabla ordenable, los ice breakers verificados y la API pública, porque ninguno es viable sobre prosa. Si se estructura, el Bloque B es la prioridad y todo lo demás se ordena detrás. No decidirlo es la peor opción, porque se sigue construyendo interfaz sobre un formato que habrá que rehacer.

Tercero, escribir lo que ya se hizo. El proyecto tiene un problema de documentación más grave que su problema de código: hay decisiones técnicas correctas —el cambio a DeepSeek, el abandono del scraping por PDL, la elección de Render— que no están registradas en ninguna parte, y hay 12 tablas diseñadas que nadie declaró como pendientes. Un documento de arquitectura actualizado y tres ADRs cambian por completo cómo se lee este proyecto desde fuera, y cuestan menos que cualquier tarea de esta lista.